Mizan

VISÃO GERAL

Arquitetura técnica do Mizan

Como interface, API, inteligência de dados e agentes trabalham no mesmo produto.

FRAMEWORK DOS AGENTES

Motor próprio em Python

Não usa LangChain nem LangGraph. Usa grafos, eventos, estado e checkpoints construídos no próprio Mizan.

Experiência — React 18 • Vite • React Router • Tailwind

Serviços — FastAPI • Pydantic • autenticação • jobs

Decisão — Orquestrador • grafos • agentes • auditoria

Dados — Supabase/Postgres • ML • integrações externas

O que é interno

EXECUÇÃO CONFIÁVEL

O Mizan define a ordem dos nós, prioridades, bloqueio de caso, idempotência, novas tentativas e transbordo para a equipe.

O que vem de bibliotecas

TECNOLOGIA DE BASE

FastAPI atende a API; React compõe a interface; Supabase guarda os dados; Gemini e Anthropic atendem chamadas de IA.

Princípio de controle

A IA gera, classifica e sugere. Regras, valores, autorização e decisões sensíveis ficam sob controle do sistema.

Mizan

MOTOR DE AGENTES

Do evento à decisão segura

O grafo mantém cada caso ordenado, recuperável e rastreável.

01
Evento
Mensagem, pagamento, vencimento, contestação ou ação humana.

02
Prioridade
A fila resolve primeiro o que exige atenção imediata.

03
Contexto
Bloqueia o caso, mantém estado e evita duplicidade.

04
Grafo
Executa triagem, regras e próximo nó condicional.

05
Resultado
Responde, registra checkpoint ou entrega à equipe.

Os papéis no fluxo

Orquestrador
Recebe eventos, controla o grafo e preserva o estado do caso.

Triagem
Classifica intenção, risco e caminho antes de agir.

Negociador
Conduz conversa, propostas permitidas e repasse humano.

Auditor
Valida política e ferramentas. A IA não autoriza ações financeiras.

Estrategista
Prioriza casos e define a abordagem de contato.

Acompanhamento
Observa promessa, vencimento, pagamento e acordo.

SUPERVISÃO E CONTINUIDADE

Falha persistente, pedido de pessoa ou situação sensível exige resumo do caso e encaminhamento para atendimento humano.

Rastreabilidade: cada passagem preserva trace ID, etapa, duração, política, tokens/custo e resultado.

Mizan

ESTRUTURAS DO PRODUTO

As camadas que sustentam o sistema

Cada parte é separada para evoluir interface, operação e inteligência sem perder controle.

Interface

- React 18
- Vite
- React Router
- Tailwind CSS
- Framer Motion
- Recharts

API e operação

- Python 3.12
- FastAPI + Uvicorn
- Pydantic
- APScheduler
- JWT e rate limit

Dados e ML

- Supabase / Postgres
- Pandas
- scikit-learn
- joblib
- Conectores de bancos

Integrações de dados

Fontes conectáveis

PostgreSQL, MySQL, SQL Server, Oracle, Google Sheets e carteiras importadas. Credenciais ficam cifradas.

Modelos e recomendações

Predição de pagamento, recuperabilidade e recomendações de estratégia usam dados e modelos versionáveis por empresa.

Segurança e observabilidade

Acesso	JWT, perfis e limite de tentativas no login.
Sigilo	Mascaramento de dados pessoais antes da IA e criptografia Fernet.
Regra	Guardrails e Auditor protegem ferramentas, valores e políticas.
Operação	Logs estruturados, trace ID, tempos, custo e registro de falhas.

Entrega

Frontend preparado para Vercel • API configurada para Render • monitoramento de disponibilidade via GitHub Actions

Mizan

LEITURA PRÁTICA

Como esta arquitetura protege a operação

O desenho técnico apoia uma cobrança mais útil, acompanhada e auditável.

1

Não perde o contexto

Cada caso carrega seu estado e checkpoints. Se um serviço de IA esperar ou falhar, o processo não precisa recomeçar do zero.

2

Não deixa a IA decidir sozinha

O modelo pode conversar e sugerir. A autorização de ferramenta, condições e ações sensíveis é conferida pelas regras e pelo Auditor.

3

Não trata toda situação igual

Triagem, estratégia e acompanhamento diferenciam uma dúvida, uma contestação, uma promessa de pagamento e uma solicitação humana.

4

Não esconde uma falha

Logs e trace ID reúnem o que ocorreu em cada etapa: tempo, custo, resultado e motivo de eventual encaminhamento.

Resumo

Mizan = aplicação React + API FastAPI + dados/ML + motor próprio de agentes orientado a eventos e políticas.

Referências no repositório: backend/agente/orquestrador.py • backend/agente/grafos.py • backend/agente/llm.py • docs/FLUXO_AGENTES.md